

California State Polytechnic University Pomona

Department of Electrical and Computer Engineering

**ECE 4301 Cryptographic Algorithms on
Reconfigurable Hardware**

**Quiz 2: SHA-1 & SHA-256 on Raspberry Pi 5 (Rust vs.
“Engine”)**

Submitted: 11/21/2025

Class Section: ECE 4301-01

Author: Dalton Hoang, Caleb Pai, Jacob Chow,

Khristian Chan, Nathan Marlow

I. Setup and Method.....	2
II. Benchmarks and Data.....	3
III. Analysis.....	4

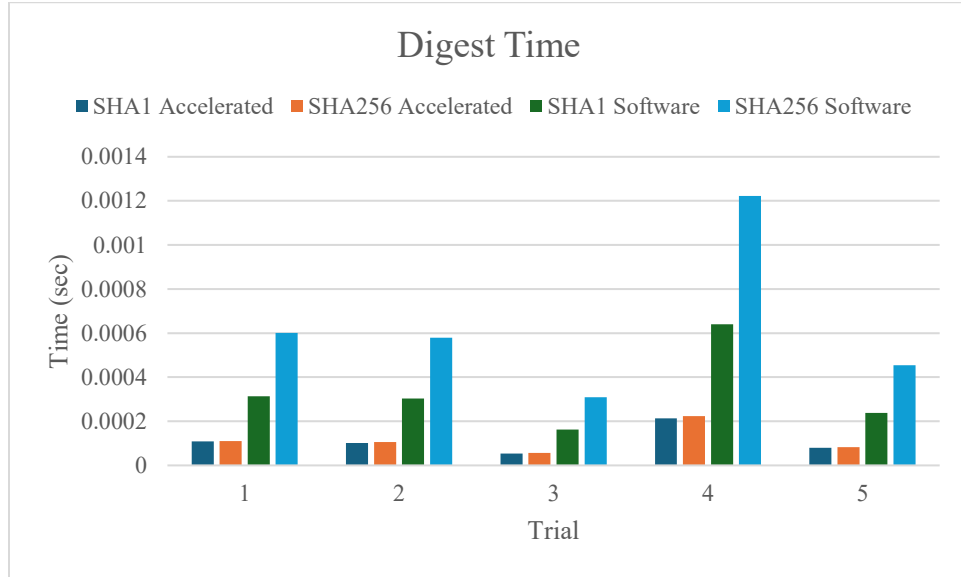
I. Setup and Method

For this project, we tested and benchmarked the SHA-1 and SHA-256 hashing functions on Raspberry Pi 5. This was done using the RustCrypto implementations of sha1 and sha2, allowing us to call the hashing logic directly in software without depending on system libraries. SHA-1 breaks the input into 512-bit blocks and then runs each block through a compression function that mixes five 32-bit words using bitwise shifts and logical operations (bitwise AND/OR/XOR) with a sequence of constant values. In this way, each block is mixed in a known but seemingly random and irreversible way, resulting in a 160-bit output, which is then used as the compression function constant for the next block. SHA-256 works similarly in that it also uses 512-bit blocks and a compression function. It differs in that each block is hashed into a 256-bit output, which goes through more rounds of processing during compression. SHA-256 uses a set of “sigma” rotation/shift functions and modular addition, making it more complex and more collision attacks.

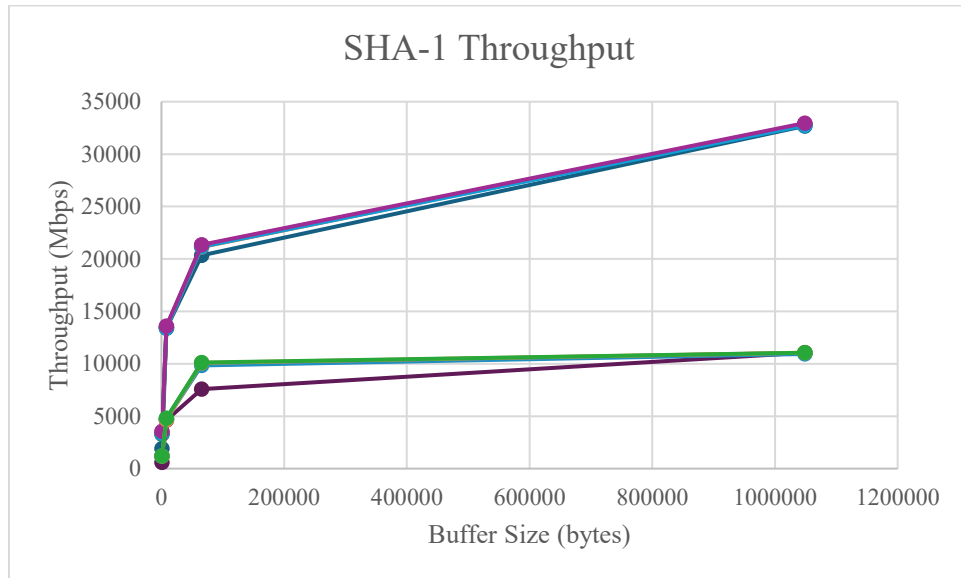
To measure the performance of both functions, we generated randomized input buffers of 1 KB, 8 KB, 64 KB, and 1 MB, and hashed them while recording the throughput. This was done for both SHA-1 and SHA-256, with five trials each. The code uses Rust’s Instant timer to bracket each hashing call and compute the effective rate based on the size of the buffer and the wall-clock time. Everything was run twice: once using a software-only binary in which the Armv8 cryptography extension was disabled, and then again using an accelerated binary which enables the Armv8 cryptography extension, which included accelerated SHA-1 and SHA-256 functions. We also pinned each run to a single core using taskset to avoid noise from other Pi processes.

For the demo, we measured the difference in digest times of the two functions using images taken by a camera on the Raspberry Pi, running the software only build and the accelerated build. These measurements let us compare the two Rust builds, which were also put up against the platform’s own hashing engines for comparison. We used kcap-speed and OpenSSL speed to measure the Pi’s built-in SHA-1 and SHA-256 performance using the same exact message sizes as our tests, which we used as a baseline to compare against. The raw outputs from these tools, along with our own results, were saved and used to generate digest time and throughput plots, as well as final comparisons between the software only build, the accelerated build, and the system hashing engines.

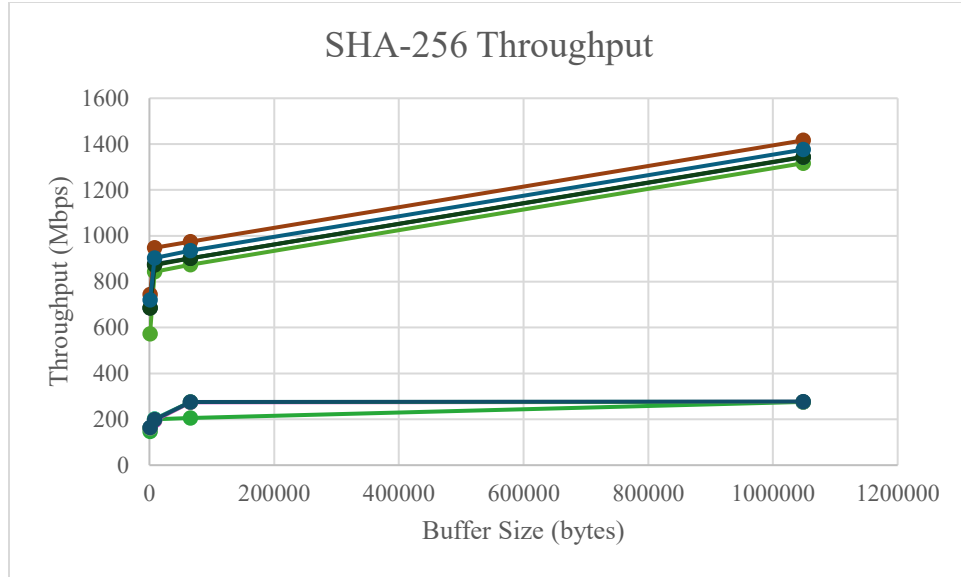
II. Benchmarks and Data



This graph shows the execution time for SHA-1 and SHA-256 using software only and accelerated Armv8 instructions across five trials.



This graph displays the throughput of SHA-1 using software only and accelerated Armv8 instructions for varying buffer sizes across five trials. The upper lines are the trials using the accelerated Armv8 instructions, and the lower lines are the trials using software only.



This graph displays the throughput of SHA-256 using software only and accelerated Armv8 instructions for varying buffer sizes across five trials. The upper lines are the trials using the accelerated Armv8 instructions, and the lower lines are the trials using software only.

III. Analysis

Successfully implementing SHA-1 and SHA-256 in Rust on the Raspberry Pi 5 and measuring the execution time and throughput provides insight on the effectiveness of the Armv8 hardware acceleration and hashing performance based on buffer size. The hardware accelerated SHA-1 and SHA-256 execute much faster, with SHA-1 being about 3 times faster and SHA-256 being more than 5 times faster than a software only implementation. The same is true of the throughput, which has similar increases when hardware accelerated. SHA-256 experiences greater performance increase than SHA-1, likely due to having more hashing rounds, allowing it to benefit more from the intrinsic instructions of the Armv8.

Looking at throughput versus buffer size, we see similar trends when comparing SHA-1 and SHA-256. While SHA-1 generally has a much higher throughput, both have a significant increase in throughput as the buffer size increases. This increase in throughput appears to be logarithmic. While this increase in throughput is extremely significant when going from a 1KB buffer to an 8KB buffer, or an 8KB buffer to a 64KB buffer, this is a much less significant increase when increasing the buffer size all the way to 1MB, showing diminishing returns towards increasing the throughput. Knowing that increased buffer sizes contribute to higher latency, it is important for buffer sizes to be chosen carefully. In real time applications where latency matters, buffer size should be as small as possible while still allowing sufficient throughput for the data stream. Smaller buffers offer greater protection against dropped or lost packets and offer generally higher security.